



FEBAUTO Racing

Competitor manual

- Part I** The platform and the rules
- Part II** From nothing to a leaderboard entry
- Part III** Learning path and reference

Formula Electric at Berkeley
September 2026, platform release v0.1.0

<https://github.com/Pranman1/feb-racing>
<https://pranman1.github.io/feb-racing/>

Contents

I	The platform	2
1	What it is	2
2	Rules	3
II	From nothing to a leaderboard entry	4
3	Install (once, about 15 minutes)	4
4	Write your driver	6
5	Score yourself and post it	8
6	Race the house racer	8
7	Enter competitions	8
III	Learning path and reference	8
8	Learning path	9
9	If something goes wrong	9
10	Command reference	10
11	Repository layout	11
A	The starter package, file by file	11
B	How the driver drives, one scan at a time	11

Part I

The platform

1 What it is

FEBAUTO Racing is a sim-racing programme for autonomous driving. Members write a ROS 2 driving stack for a 1:10 scale RoboRacer car, run it in the FEB Simulator on their own laptop, and post lap times to a public leaderboard. Twice a semester the organiser runs a competition on a secret track, with the same rules as the RoboRacer Sim Racing League.

The simulator is a fork of the AutoDRIVE Simulator (Unity). The car, its sensors and its physics are the league's RoboRacer model, unchanged, so a stack that works here works in the league too.

1.1 The pieces

Piece	What it does	Who touches it
FEB Simulator	Native app (Mac, Windows, Linux). Simulates the car on a track folder and talks to a bridge over a WebSocket on port 4567.	Everyone runs it, nobody edits it
Devkit container	Docker image <code>ghcr.io/pranman1/feb-devkit</code> : ROS 2 Humble plus the AutoDRIVE bridge that turns the WebSocket into ROS 2 topics. The member's code runs inside it.	Members' code runs inside it
<code>stack/</code>	The member's ROS 2 packages. Mounted into the container and built on every start.	Members
<code>tracks/</code>	Public tracks. PNG and YAML in, <code>track.json</code> out. Shipped inside the app.	Organiser adds, members use
<code>feb-sim</code>	The member launcher: <code>setup</code> , <code>run</code> , <code>practice</code> , <code>submit</code> , <code>shell</code> , <code>stop</code> .	Members
<code>feb-race</code>	The organiser harness: events, nightly reruns, verification, head-to-head brackets.	Organiser
<code>results/</code> , <code>events/</code> , <code>submissions.yaml</code>	The data behind the leaderboard site.	Harness writes, members' pull
Site	GitHub Pages, rebuilt on every push to <code>main</code> .	requests add Read-only

1.2 Data flow while driving

```
lidar, imu, encoders, camera, lap/collision counts --> bridge --> /autodrive/roboracer_1
/* --> your node
your node --> /autodrive/roboracer_1/steering_command, throttle_command --> bridge -->
simulator
```

The car takes exactly two inputs and everything else is a sensor:

- `steering_command`, `std_msgs/Float32` in $[-1, 1]$: full left to full right, ± 0.5236 rad. The actuator lags about 0.25 s and slews at 3.2 rad/s.
- `throttle_command`, `std_msgs/Float32` in $[-1, 1]$. **Zero is a hard brake**, not coasting. A car that wants to keep moving must never send zero.

1.3 Sensors (namespace /autodrive/roboracer_1/)

Topic	Type	Notes
lidar	sensor_msgs/LaserScan	1080 beams over 270°, 0.06 to 10 m, 40 Hz
imu	sensor_msgs/Imu	orientation, angular velocity, linear acceleration
left_encoder, right_encoder	sensor_msgs/JointState	wheel angle in rad; wheel radius 0.059 m
steering, throttle	std_msgs/Float32	actuator feedback
front_camera	sensor_msgs/Image	forward camera (cone tracks)
ips, odom, tf	pose	ground truth: training and debugging only
lap_count, lap_time, last_lap_time, best_lap_time, collision_count	counters	scoring: training and debugging only
reset_command	command	respawn: training and debugging only

Wheelbase 0.324 m, half width 0.135 m, steady-state speed about 23 m/s per unit throttle. With a window open the bridge exchanges data at about 10 Hz; headless it reaches 20 to 40 Hz. Scored runs go through a proxy fixed at 10 Hz for everyone, so a driver must cope with 10 Hz. The starter kit does.

1.4 Time

Physics runs in *simulated* time. When the machine cannot keep up (a screen recorder, a browser, two cars) the simulator advances physics by at most 0.1 s per frame, so simulated time falls behind the wall clock rather than the car jumping. Everything is built so that this cannot hurt a driver: every message stamp and the /clock topic carry simulated time, the starter kit runs with `use_sim_time`, lap timers count simulated time, the scoring proxy delivers 10 Hz of *simulated* time, and the render rate is capped so it rarely happens at all. The HUD shows the real-time factor (RTF, simulated seconds per wall second; red below 0.9) and every result records it. If you write your own nodes, use the message stamps or `use_sim_time`, never the wall clock, for speeds and integrals.

2 Rules

These follow the RoboRacer Sim Racing League.

- One attempt is 1 warm-up lap plus 10 timed laps. Total time is the sum of the timed laps plus **10 s per collision**.
- Not finishing within **300 s** is DNF, ranked after every finisher by laps completed. More than **10 collisions** is DSQ.
- Each team gets the number of attempts stated in the event; the best counts.
- **Qualifying for the first competition:** on *every* practice track, a verified practice run of 10 laps with at most 1 collision. Practice tracks are added through the semester; the bar applies to all of them. The Teams page of the site shows each team's progress per track.
- **The final** is head-to-head, two cars at a time, in a knockout seeded by the qualification standings; only the top seeds (set per event, 8 by default) enter.
- **Restricted topics at race time:** `ips`, `odom`, `tf`, the lap and collision counters, and `reset_command`. The harness records which nodes subscribe to them during the run and flags the result on the site. Use them freely while developing.
- Everything a submission needs starts from the container's entrypoint (`/home/autodrive_devkit.sh`), never from `.bashrc`.
- Head-to-head: car-to-car contact counts as a collision for both cars and respawns them side by side. Finishing order is most laps first, then the finish time including penalties.

Part II

From nothing to a leaderboard entry

3 Install (once, about 15 minutes)

3.1 Prerequisites

OS	Install
macOS	Docker Desktop (https://docker.com), Python 3 (comes with the Xcode command line tools: <code>xcode-select -install</code>), GitHub CLI (<code>brew install gh</code>), Foxglove (the .dmg from https://foxglove.dev/download)
Windows	Docker Desktop with the WSL 2 backend, Python 3 from python.org, GitHub CLI (<code>winget install GitHub.cli</code>), Foxglove (the installer from https://foxglove.dev/download). Run the commands below in Git Bash or WSL.
Linux	<code>sudo apt install docker.io python3 gh</code> , then add yourself to the <code>docker</code> group and log in again; Foxglove: <code>sudo apt install ./foxglove-studio-*.deb</code> from https://foxglove.dev/download

Start Docker Desktop before anything else. Log the GitHub CLI in once: `gh auth login`. Open Foxglove once and sign in (free account); it is how you look at your car's data, see section 4.4. Mac users: `competitor_mac.pdf` is this manual's install chapter with every click spelled out, including the Rosetta and Docker Desktop details.

Windows, step by step. The launcher is a Python script run from a Unix-style shell; Git Bash is the simplest.

1. Open PowerShell as administrator and run `wsl -install`. Reboot when it asks. This turns on WSL 2, which Docker Desktop needs.

2. Install Docker Desktop from <https://docker.com>. In its settings, General, keep “Use the WSL 2 based engine” ticked. Start it and wait for the whale icon to stop animating.
3. Install Python 3 from <https://python.org>; on the first installer page tick “Add python.exe to PATH”.
4. Install Git for Windows from <https://git-scm.com>; it brings Git Bash. Every `./feb-sim` command in this manual is typed in a Git Bash window.
5. `winget install GitHub.cli`, then in Git Bash `gh auth login`.
6. Install Foxglove from <https://foxglove.dev/download>, open it once, sign in.

The simulator is a native Windows build; it needs a GPU with up-to-date drivers and, on a laptop with two GPUs, run it on the discrete one (Windows Graphics settings, add the app under `%USERPROFILE%\AppData\Local\Programs\Foxglove Studio`).

Linux, step by step (Ubuntu 22.04 or 24.04; other distributions differ only in the package commands).

1. `sudo apt install docker.io python3 git curl unzip`.
2. `sudo usermod -aG docker $USER`, then log out and back in so `docker ps` works without `sudo`.
3. GitHub CLI: on 24.04 `sudo apt install gh`; on 22.04 follow https://github.com/cli/cli/blob/trunk/docs/install_linux.md (three lines), then `gh auth login`.
4. Foxglove: download the `.deb` from <https://foxglove.dev/download> and `sudo apt install ./foxglove-studio-*.deb`; open it once, sign in.

The simulator needs Vulkan: Intel and AMD GPUs work with the stock Mesa drivers, NVIDIA needs the proprietary driver. Run from a desktop session, not over plain SSH.

3.2 Get the repo and the simulator

```
git clone https://github.com/Pranman1/feb-racing
cd feb-racing
./feb-sim setup --team "Your Team"
```

`setup` checks Docker, pulls the devkit image, downloads the simulator app for your OS into `~/feb-sim/app/`, and checks that Foxglove is installed (it opens the download page if not). The team name is what your results are posted under; use the name your team registers in `submissions.yaml`.

3.3 First drive: manual

```
./feb-sim run
```

A window opens (after the FEBAUTO Sim splash) on the *FEB Loop* track in Driver’s Eye view. Nothing drives by itself: with no stack given, the launcher starts no driver and no opponent. Arrow keys steer and throttle. The menu in the top-left has the Manual/Autonomous button, the Track button (cycles the built-in tracks), the Cars button (more cars on the grid), the Look button (Visual or Simple scene; Simple for a weak laptop) and the camera choices. The HUD at the bottom shows the running lap time (its clock starts when you first cross the start line), last and best lap, lap count, collisions and speed. After your first completed lap a translucent gold ghost replays your best lap so you can see where you gain and lose time.

macOS, if the app does not open (“damaged” or “cannot be opened”): the app is not signed by Apple. Run once:

```
xattr -dr com.apple.quarantine ~/.feb-sim/app/mac/"FEB Simulator.app"
codesign --force --deep -s - ~/.feb-sim/app/mac/"FEB Simulator.app"
```

4 Write your driver

4.1 Where the code lives

Copy the starter kit into your own folder. `stack/` is yours and is ignored by git in this repo, so nothing you do there conflicts with updates.

```
cp -r starter_kit/feb_driver stack/
```

<code>stack/feb_driver/</code>	
<code>feb_driver/reactive_driver.py</code>	THE DRIVER. Subscribes to lidar + encoders,
<code>publishes steering + throttle</code>	
<code>config/driver.yaml</code>	its tuning knobs (speeds, gap bubble, filters,
<code>throttle law)</code>	
<code>launch/drive.launch.py</code>	starts the driver with the yaml
<code>feb_driver/sysid_probe.py</code>	stage 2: drives test inputs and records a bag
<code>tools/sysid_fit.py</code>	stage 2: fits the recorded bag
<code>package.xml, setup.py</code>	standard ROS 2 Python package files

`reactive_driver.py` is about 170 lines. Per lidar scan it cleans the scan, zeroes a safety bubble around the nearest return, aims into the widest gap, picks a speed from the room ahead and the steering angle, and turns that into a throttle as feed-forward plus a bounded trim. Everything is commented. `driver.yaml` exposes every number.

You can add more packages next to `feb_driver`, more nodes, C++ packages, anything that builds with `colcon`. The container builds everything under `stack/` on start.

4.2 Run it on the simulator

```
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py"
```

What happens, in order: the launcher starts the devkit container with `stack/` mounted inside it; the container builds your packages and starts the bridge; the simulator starts, connects to the bridge, and is put in Autonomous mode; the container starts your launch command. The car drives.

Edit, save, run the same command again. Each run is a fresh build (a few seconds for Python).

4.3 Useful options

Option	Effect
<code>-track porto</code>	another built-in track (<code>loop</code> , <code>loop_cones</code> , <code>porto</code>), or a path to any track folder
<code>-mode manual</code>	keep the keyboard even with a stack running
<code>-opponent normal</code>	the house racer in car two (<code>slow 1.5</code> , <code>normal 2.0</code> , <code>fast 2.5 m/s</code> , or <code>map</code>). Needs <code>-stack</code> . Not on Porto (single-car track).
<code>-cars 2</code>	a second, undriven car on the grid
<code>-noise 1</code>	lidar range noise and dropouts; <code>-noise "range_sigma:=0.05 latency:=0.05"</code> sets them
<code>-lidar-hz 10</code>	slower lidar, as a real bridge often delivers
<code>-camera Trackcam</code>	start in another view ("God's Eye", <code>Trackcam</code> , "Driver's Eye")
<code>-no-foxglove</code>	do not open Foxglove on the car's topics
<code>-fps 30</code>	render-rate cap (default 60, 0 uncapped); lower it on a weak laptop so simulated time keeps pace
<code>-camera-hz 5</code>	how often the bridge reads the camera back (default 10); the costly part of each message
<code>-look simple</code>	the bare scene instead of the dressed one (<i>Visual</i> : asphalt, banded walls, start line, sky, livery). Purely cosmetic; the menu's Look button toggles it and the choice is remembered.

Car one is always *your* car: the Manual/Autonomous button, the keyboard, the Driver's Eye camera and topic namespace `roboracer_1` all belong to it. Your container never sees any other car.

4.4 See the data: Foxglove

The devkit runs a Foxglove bridge on port 8765, so the Foxglove app on your laptop shows every topic live on any OS, with no ROS installed outside the container. With the app installed and signed in (prerequisites above), `./feb-sim run` opens Foxglove connected to your car, a few seconds after the simulator window. Without the app installed the launcher prints the address instead: `ws://localhost:8765` under *Open connection, Foxglove WebSocket*.

Useful panels: 3D (lidar as `/autodrive/roboracer_1/lidar`), Image (the front camera), Plot (`throttle_command`, `steering_command` and the encoders against time), Raw Messages and the Topics list with rates. Foxglove also opens the bags that `practice` records under `runs/<id>/`. `-no-foxglove` skips the auto-open.

4.5 Debug

```
./feb-sim shell          # a ROS 2 shell inside the running container
ros2 node list
ros2 topic hz /autodrive/roboracer_1/lidar
ros2 topic echo /autodrive/roboracer_1/throttle_command
rqt
./feb-sim stop           # stop the container(s) and the simulator
```

The container log (build errors, your node's output) is `docker logs feb-devkit`.

4.6 Keeping up to date

```
./feb-sim update
```

One command pulls the repo (new practice tracks, launcher and docs), pulls the newer devkit image if there is one, and replaces the simulator app if the organiser has released a new build. New tracks need no app update: the launcher hands the repo's `tracks/` folder to the app, so a pulled

track appears in the Track button right away. `./feb-sim run` prints one line when a newer app exists.

5 Score yourself and post it

```
./feb-sim practice --stack "ros2 launch feb_driver drive.launch.py" --track loop
./feb-sim submit
```

`practice` runs one attempt under competition rules: warm-up plus 10 timed laps, 10 s per collision, DNF after 300 s, through the 10 Hz proxy. It prints the result and saves `runs/<id>/` with `result.json`, logs and a ROS bag.

`submit` opens a pull request with the result file. A check validates it and merges it, and you appear on that track's practice board as *unverified*. The organiser's nightly rerun of your image turns it *verified*.

6 Race the house racer

```
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py" --opponent normal
```

The house racer starts in car two only when your stack starts your car. It is the smooth lidar driver at a conservative pace with zero wall contact: the thing a new stack has to overtake. It follows a car ahead rather than ramming it. `-opponent map` runs the map-following driver instead (it knows the track and the car's true pose; faster, less smooth). On cone-only tracks the launcher always uses the map follower, since a gap follower has no walls to follow there.

7 Enter competitions

The season has three steps. First, **qualify**: complete 10 laps with at most one collision on every practice track, verified by the organiser's nightly rerun of your image (so the image has to be registered, below). The Teams page of the site shows a tick per track and a *qualified* badge when all are done. Second, the **qualification competition**: a time-attack on a secret track, open to qualified teams. Third, the **final**: a head-to-head knockout, two cars per race, for the top seeds of the qualification.

Competitions run on the organiser's PC from a Docker image you publish. Build it from the starter Dockerfile, which starts from the devkit image, copies `stack/` in, builds it and sets the launch command:

```
docker build -t <dockerhub-user>/feb-racer:v1 -f starter_kit/Dockerfile stack/
docker push <dockerhub-user>/feb-racer:v1
```

If your launch command differs from the starter's, change the `FEB_LAUNCH` line in a copy of the Dockerfile. Then register: add your team, GitHub logins and image to `submissions.yaml` in a pull request:

```
teams:
- name: Your Team
  members: [github-login-1, github-login-2]
  image: docker.io/<dockerhub-user>/feb-racer:v1
```

Competitions and nightly reruns pull that image. Update the tag and the yaml whenever you want a new version raced. In a head-to-head final your container runs unchanged: a proxy between the simulator and the containers makes every car look like `roboracer_1` to its own stack.

Part III

Learning path and reference

8 Learning path

Every stage ends with a number on the leaderboard.

1. **Read the car.** `./feb-sim shell`, then `ros2 topic hz` and `ros2 topic echo` on the topics in Part I. Steering lags about 0.25 s; throttle 0 brakes hard.
2. **Reactive driving.** Tune `driver.yaml`: bubble, gap field of view, the deep-beam weight, speed limits, braking margin, filter time constants. Target: 10 clean laps on `loop` under 100 s.
3. **System identification.** `ros2 launch feb_driver sysid.launch.py` drives throttle steps, a coast and a chirp while recording a bag into `stack/sysid_bag`. Inside the container, `python3 tools/sysid_fit.py stack/sysid_bag` gives the steady-state gain (m/s per throttle), the idle-brake deceleration and the throttle and steering lags. Put the gain into `speed_per_throttle`. Expect the tyres to be a knife edge: lateral grip peaks at 1 % slip and halves by 10 %.
4. **Localisation and a map.** Build a map of the track from lidar (scan matching, or SLAM Toolbox in the container), then localise on it at race time. Check against `ips` while developing; never subscribe to it at race time.
5. **Raceline and controller.** A minimum-curvature line on your map with a speed profile from your sysid numbers, tracked by pure pursuit, then MAP or MPC. Lap times halve here.
6. **Realism.** `-noise`, `-lidar-hz 10`, `-opponent`, and the cone track (`loop_cones`: blue cones left, yellow right, a camera perception project).

9 If something goes wrong

Symptom	Cause and fix
Nothing drives	Correct without <code>-stack</code> . With a stack: the build failed or the node crashed, see <code>docker logs feb-devkit</code> ; or the mode button says Manual.
Port 4567 in use	Another bridge container is running: <code>./feb-sim stop</code> .
Car surges and stops	Throttle reaching zero (a hard brake). Keep feed-forward plus a bounded trim; see the starter's throttle law.
Car weaves at full lock	Raw beam-to-beam targets at 10 Hz. Low-pass and rate-limit steering; the starter shows how.
Docker pull denied	Docker Desktop not running, or the image is private; ask the organiser.
Mac app will not open	See the <code>xattr</code> and <code>codesign</code> commands in the install section.
Cones or walls missing	The track folder has no <code>track.json</code> ; run <code>tools/track_build.py <folder></code> .

10 Command reference

Command	What it does
<code>feb-sim setup -team T</code>	pull the devkit image, download the app, set the team name
<code>feb-sim run [-track] [-stack] [-mode] [-opponent] [-cars] [-noise] [-lidar-hz] [-camera] [-no-foxglove]</code>	start the container, the simulator and Foxglove, connected
<code>feb-sim practice -stack S [-track] [-laps]</code>	one scored attempt into <code>runs/<id>/</code>
<code>feb-sim submit [-run DIR]</code>	pull request with the newest (or given) result
<code>feb-sim update</code>	git pull, newer devkit image, newer simulator app
<code>feb-sim shell</code>	ROS 2 shell inside the running container
<code>feb-sim stop</code>	stop containers and the simulator

11 Repository layout

```

feb-racing/
  feb-sim           the launcher (Python 3, needs only docker and gh)
  starter_kit/      feb_driver package + submission Dockerfile
  stack/            YOUR code (gitignored)
  tracks/           public tracks (also shipped inside the app)
  runs/             your attempt folders (gitignored)
  results/, events/, submissions.yaml, site/   leaderboard data and page
  docs/            handbook, this manual, detail pages

```

A The starter package, file by file

`starter_kit/feb_driver` is a normal ROS 2 Python package. After `cp -r starter_kit/feb_driver stack/` these are the files you own (the Python files live in the inner `feb_driver/` folder):

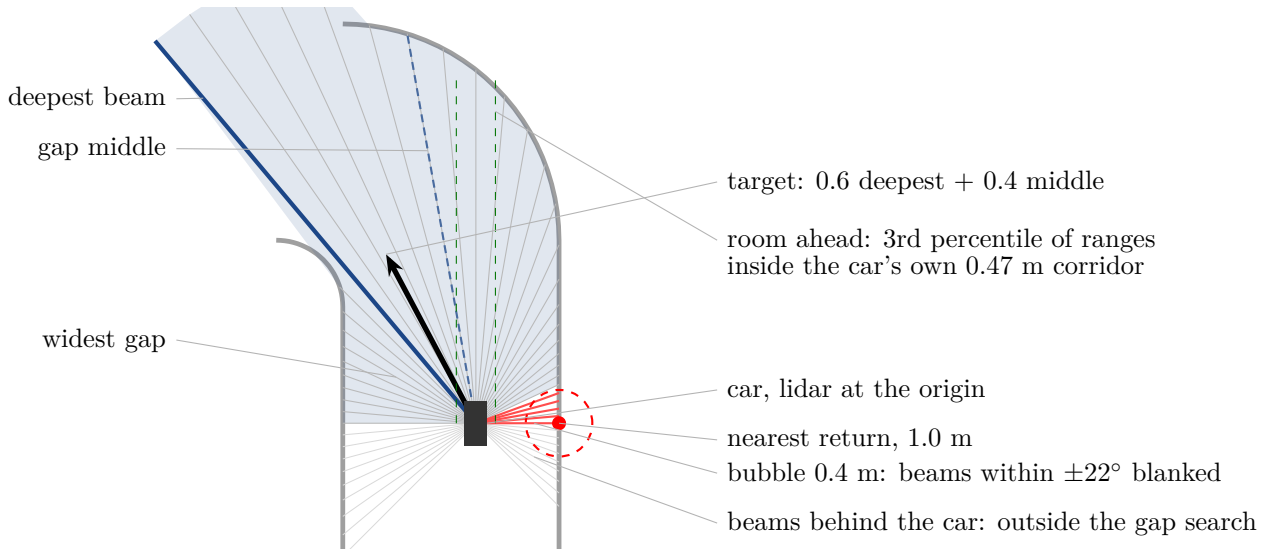
File	What it is
<code>package.xml</code>	The ROS 2 manifest: the package name <code>feb_driver</code> , its version and its dependencies (<code>rcipy</code> , <code>sensor_msgs</code> , <code>std_msgs</code>). <code>colcon</code> reads it to know the package exists. You only touch it to add a dependency.
<code>setup.py</code>	Python packaging. The line that matters is the entry point <code>reactive_driver = feb_driver.reactive_driver:main</code> : it is what makes <code>ros2 run feb_driver reactive_driver</code> exist. It also installs <code>launch/</code> and <code>config/</code> so launch files can find them. Add a line here for every new node.
<code>setup.cfg</code>	Two lines telling <code>colcon</code> where to put the installed scripts. Never edited.
<code>resource/feb_driver</code>	An empty marker file ROS 2 uses to index the package. Never edited.
<code>__init__.py</code>	Empty; makes the folder a Python module.
<code>reactive_driver.py</code>	The driver, about 170 lines, explained below.
<code>config/driver.yaml</code>	Every tuning number, one per line with a comment. The launch file hands it to the node, so you change behaviour here without touching code.
<code>launch/drive.launch.py</code>	Starts one node, <code>reactive_driver</code> , with the <code>yaml</code> and <code>use_sim_time</code> set, so the node's clock follows the simulator. This is what <code>-stack "ros2 launch feb_driver drive.launch.py"</code> runs.
<code>sysid_probe.py</code> , <code>launch/sysid.launch.py</code>	Stage 2. Drives with the same steering but a fixed throttle schedule (three steps, a coast, a chirp) while recording a bag, to measure how the car responds.
<code>tools/sysid_fit.py</code>	Fits that bag: steady-state gain (the <code>speed_per_throttle</code> number), idle brake, throttle lag and effective wheelbase. Run inside the container.

The node subscribes to three topics and publishes two, all under `/autodrive/roboracer_1/`: in, `lidar` (`LaserScan`, 1081 ranges over $\pm 135^\circ$) and `left_encoder`, `right_encoder` (`JointState`, wheel angle); out, `steering_command` and `throttle_command` (`Float32`). Steering is normalised, -1 to 1 meaning -30° to 30° . Throttle is 0 to 1 and **0 is a hard brake**, not coasting.

B How the driver drives, one scan at a time

Every lidar scan, about ten times a second, `on_scan` runs the steps below and publishes one steering and one throttle value. The figure shows one scan from above, in a corridor that bends left; the

right wall is the nearest thing the lidar sees.



1. **Clean the scan.** Any beam that returned nothing (infinity) is set to the lidar's maximum range, and every range is clipped to the sensor's limits. This is the numpy array `ranges`; `angles` is the matching bearing of each beam.
2. **Find the nearest return and blank a bubble around it.** Only beams within $\pm \text{gap_fov}$ (1.6 rad, about $\pm 92^\circ$) count. The shortest of them is the nearest obstacle, the red dot. Every beam whose bearing is within $\arctan(\text{bubble}/r_{\min})$ of that beam is set to zero: at $r_{\min} = 1.0$ m and `bubble` = 0.4 m that is $\pm 22^\circ$, the red beams. A zeroed beam can never be chosen as a direction, so the car never aims through a gap that passes within 0.4 m of the closest thing it sees. That is the whole safety idea, and it is why the car brushes walls when `bubble` is too small and refuses corners when it is too big.
3. **Pick the widest gap.** The non-zero beams are split into runs of consecutive beams; the longest run is the gap (blue). Inside it, two candidates: the deepest beam (the longest range) and the middle beam of the run. The target bearing is a blend, `deep_weight` $\times$ deepest + $(1 - \text{deep_weight}) \times$ middle, 0.6/0.4 by default. Pure deepest cuts corners tight and hunts; pure middle is smooth but wide.
4. **Turn the bearing into steering.** The car is still turning at its current steering angle for the actuator delay (0.25 s) plus one scan period before the new command takes effect, so the heading change over that time is subtracted from the target first (delay compensation). Then `steer_gain` scales the bearing (0.7: a 20° target asks for 14° of steering), the result is clipped to the $\pm 30^\circ$ limit, low-passed with `steer_tau` and rate-limited to the actuator's 3.2 rad/s. Without the filtering, a target that jumps from one beam to the next would flick the wheel full lock every scan.
5. **Measure the room ahead.** Take the beams whose end point lies inside the car's own corridor (a strip 0.235 m either side of the centreline, the green dashes). The 3rd percentile of their forward distance is the clearance: not the minimum, which one noisy beam would spoil, but close to it. It is low-passed, except that a genuinely close obstacle (under `brake_margin` + 0.3 m) passes straight through so braking is immediate.
6. **Pick a speed.** Three ceilings: `max_speed` (2.5 m/s); the cornering speed $\sqrt{\text{lat_accel}/\kappa}$ from the slowly filtered steering, where $\kappa = \tan(\delta)/L$ is the path curvature; and the braking speed $\sqrt{2 \text{decel_limit} (\text{clearance} - \text{brake_margin})}$, the fastest speed from which the car can still stop before the clearance. The goal is the smallest of the three, floored at `min_speed` (0.9 m/s) so

the car never parks in a corner. The goal is low-passed and then slewed at `accel_limit` up and `decel_limit` down.

7. **Speed to throttle.** The wheel encoders give the measured speed (wheel angle change over the last 0.25 s times the wheel radius). Throttle is feed-forward plus trim: feed-forward is $v_{\text{target}}/\text{speed_per_throttle}$ (23 m/s per unit throttle, measured with the sysid tools, so 1 m/s needs about 0.043); trim is a small PI on the speed error, clipped to half of the feed-forward. Because of that clip the command can never reach zero while speed is wanted, and zero would be a hard brake. Finally the throttle is slew-limited to 0.8 per second. A textbook PI speed loop without these two guards surges and brakes in a 0.8 s cycle on this simulator; this one holds speed at 10 Hz.
8. **Publish.** Steering divided by 30° goes to `steering_command`, the throttle to `throttle_command`. On shutdown the node publishes 0,0 once so the car stops.

Follow mode (`follow_gap` > 0, used by the house racer as the opponent): if something sits within that distance in a $\pm 15^\circ$ cone straight ahead, it is treated as a car to sit behind, not a wall to steer around. Its beams are replaced by the corridor seen beside it, so the steering keeps the corridor's line, and the speed goal is capped to hold the distance.

Where to start tuning. `bubble` and `deep_weight` decide the line; `max_speed`, `lat_accel` and `brake_margin` decide how fast it dares. Change one number, run the same command, watch the lap time. The sysid tools are for when you want the throttle law to match the car rather than guess.